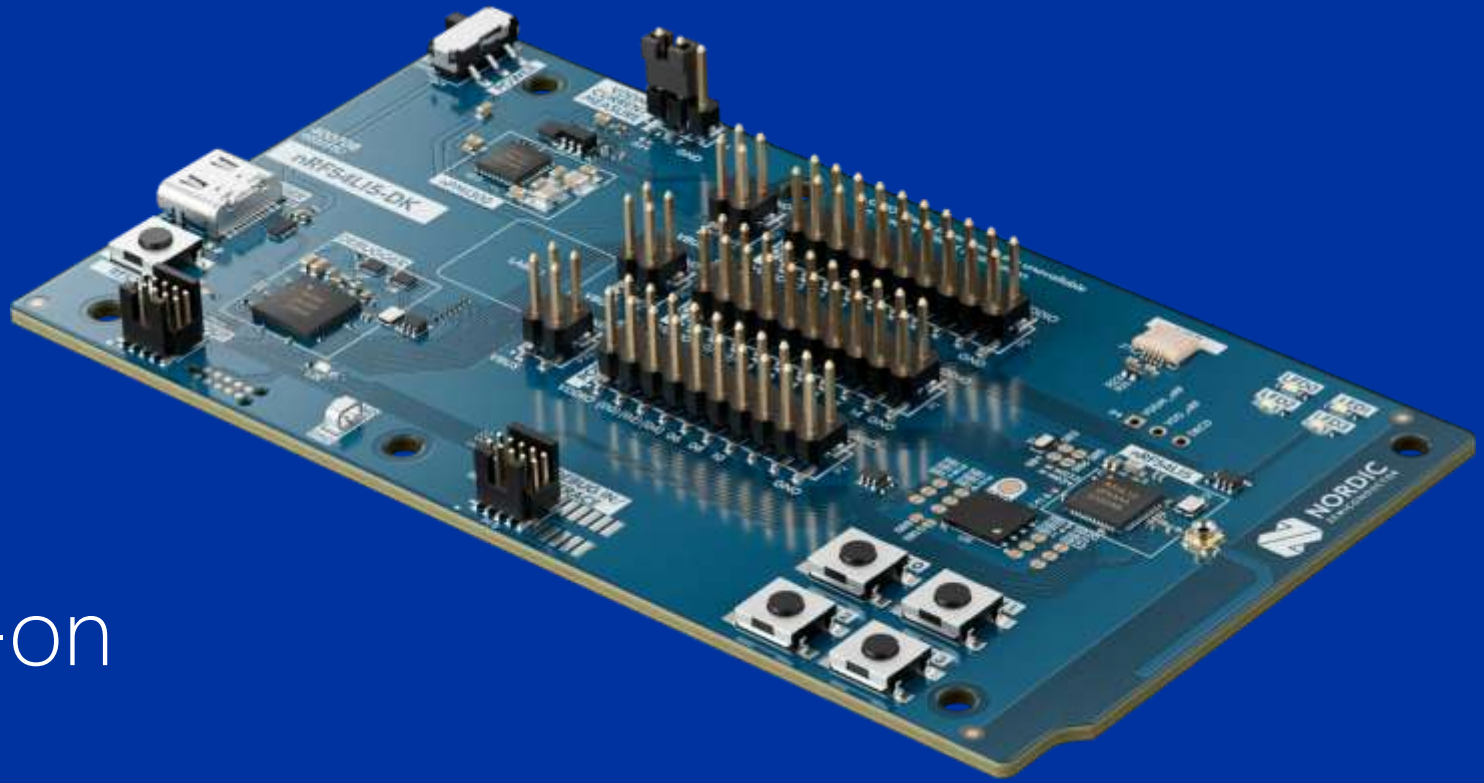
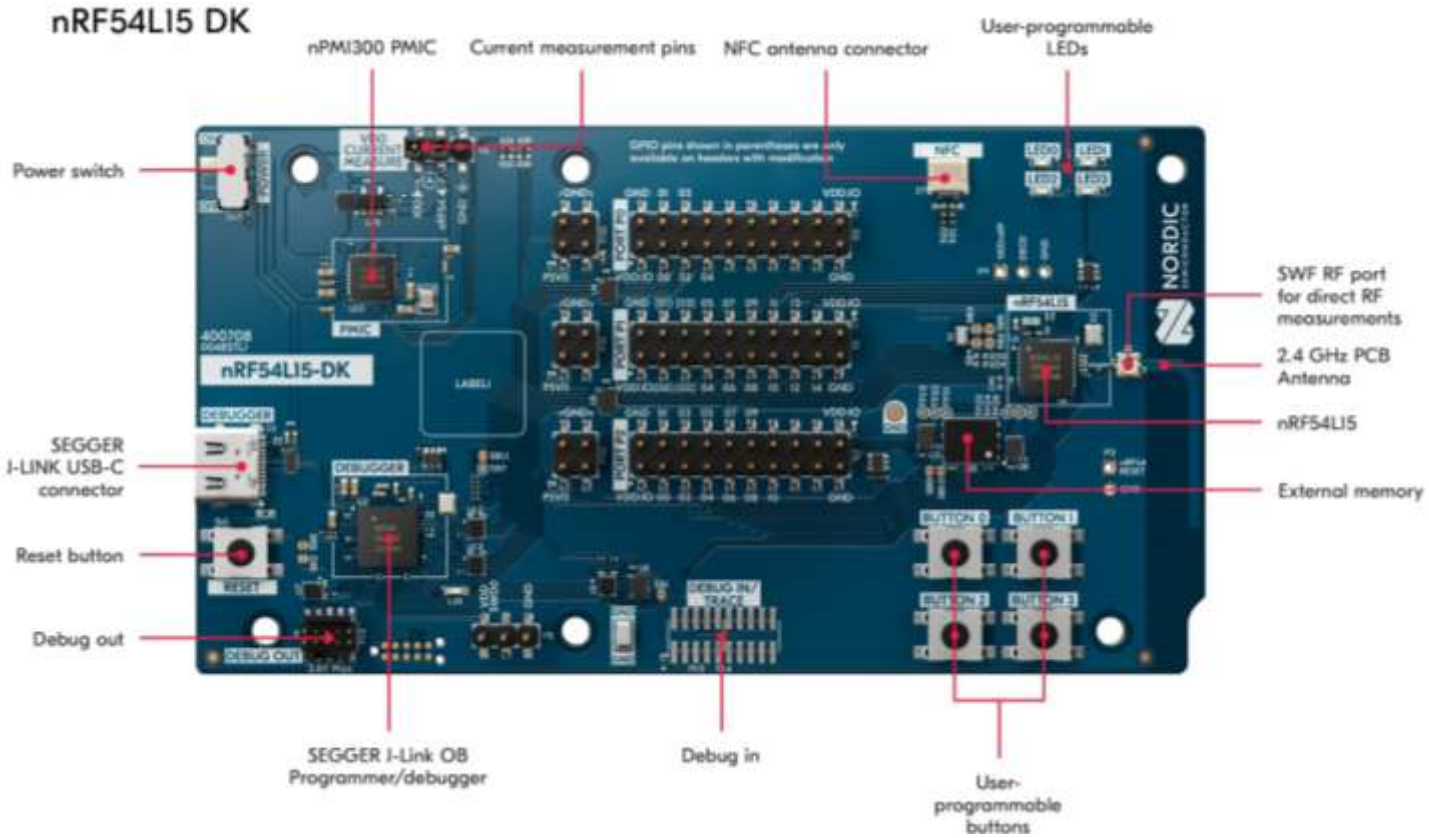


Hands-on

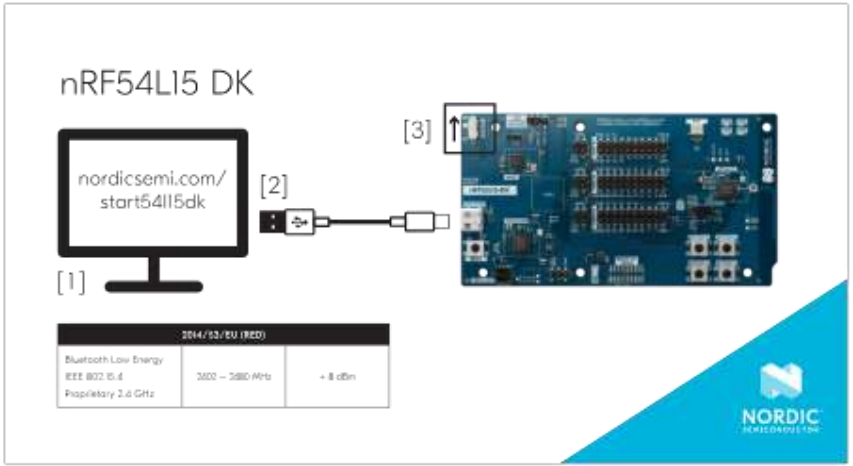
Copyright © 2025 Nordic Semiconductor. All rights reserved

Hands-on: Unboxing nRF54L15DK



Copyright © 2025 Nordic Semiconductor. All rights reserved

Get started with the nRF54L Series



Copyright © 2025 Nordic Semiconductor. All rights reserved

Get started with the nRF54L15 DK

Step 1 of 3

Download and install nRF Connect for Desktop

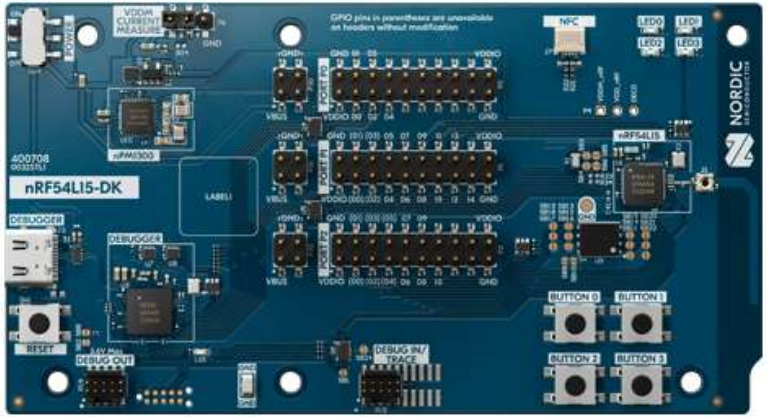
→ [Go to nRF Connect for Desktop]

Copyright © 2025 Nordic Semiconductor. All rights reserved

Get started with the nRF54L15 DK

Step 2 of 3

Connect your nRF54L15 DK to your PC or Mac with a USB cable and power on the DK

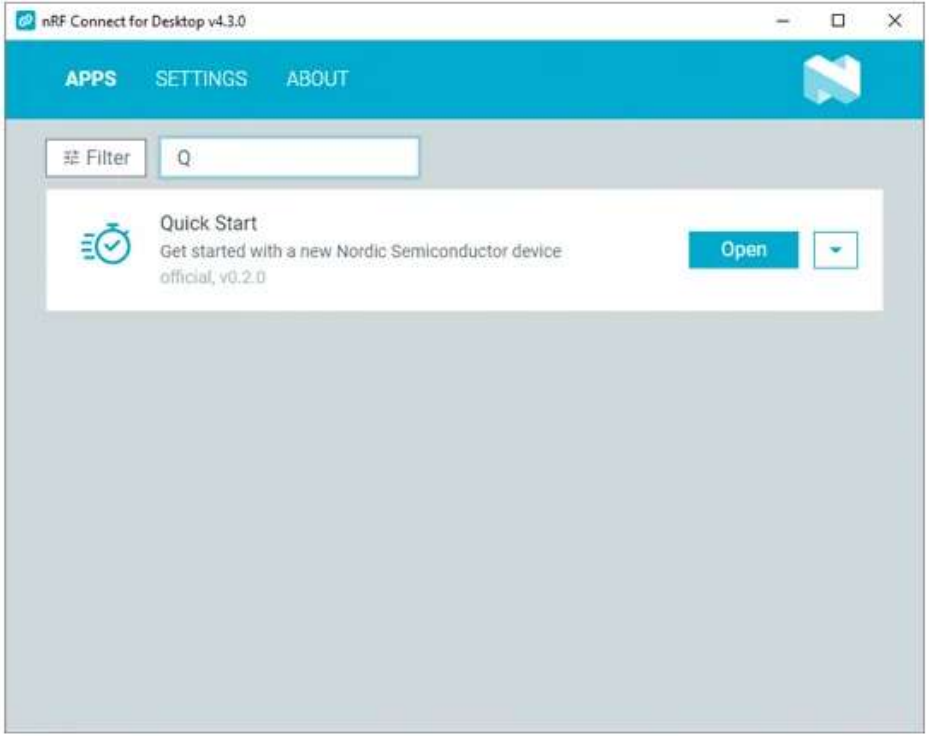


Copyright © 2025 Nordic Semiconductor. All rights reserved

Get started with the nRF54L15 DK

Step 3 of 3

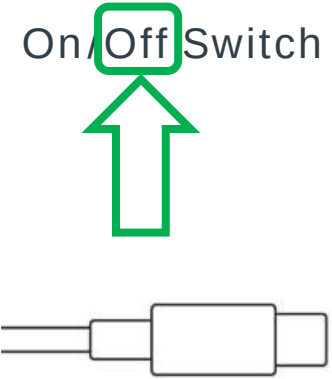
Open Quick Start in nRF Connect for Desktop and follow the instructions in the app



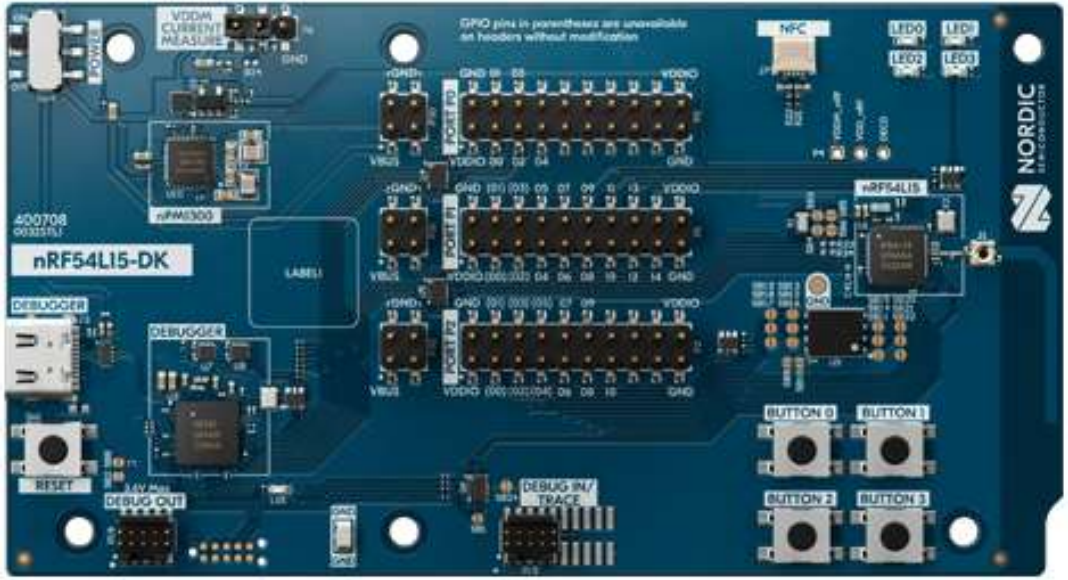
Copyright © 2025 Nordic Semiconductor. All rights reserved

Hands-on: Connect Sensor Board

On/Off Switch

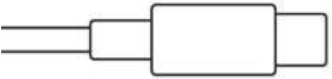


USB cable to PC

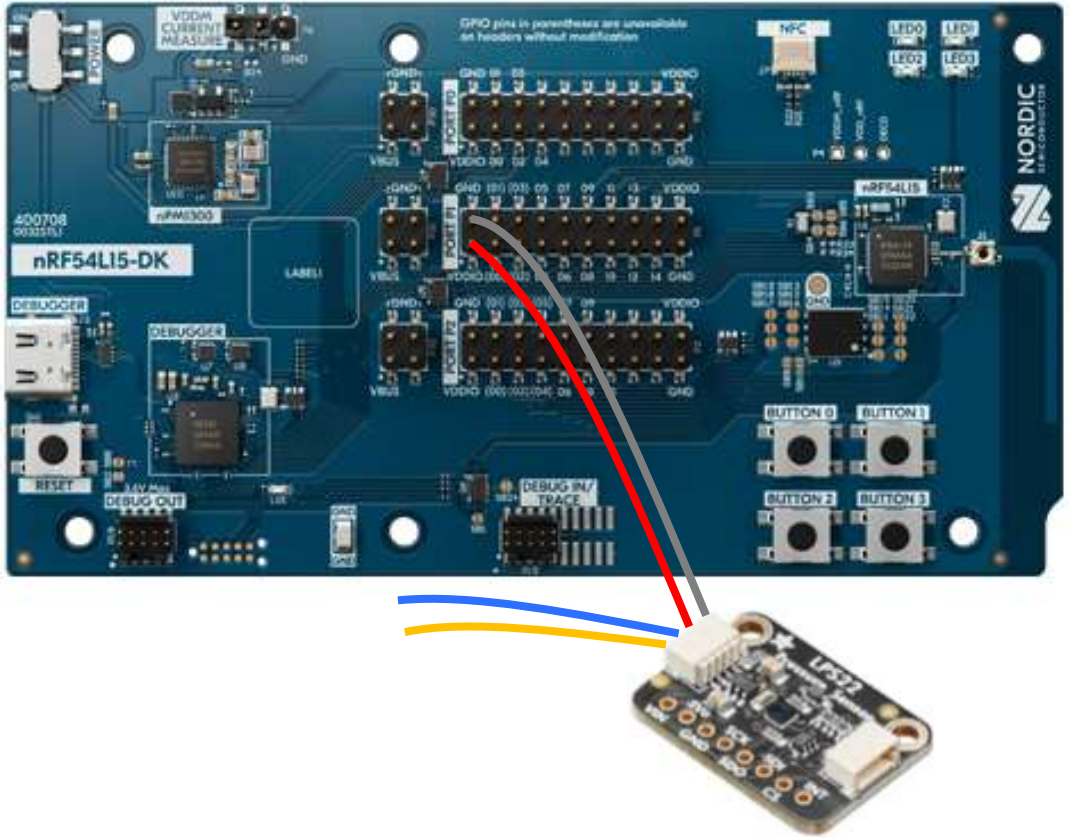


Hands-on: Connect Sensor Board

On/Off Switch



USB cable to PC



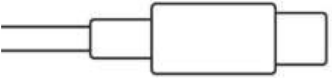
GND

VDDIO

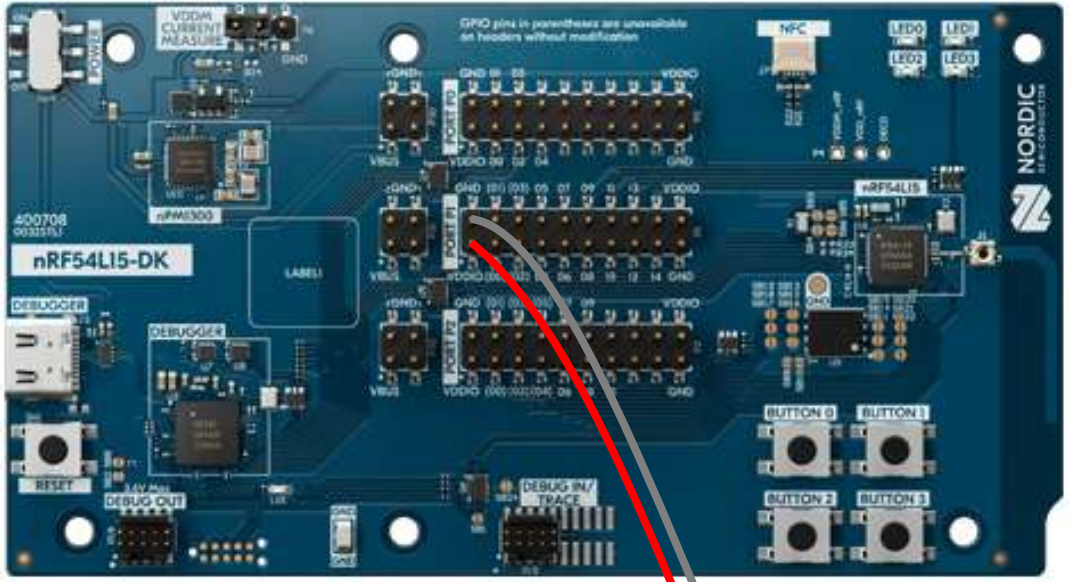
Unconnected!!!

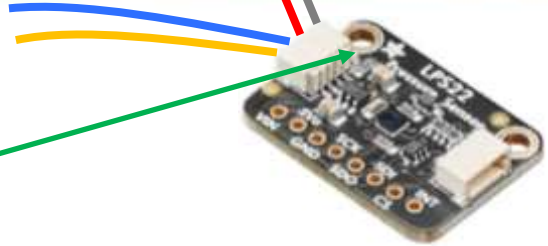
Hands-on: Connect Sensor Board

On/Off Switch



USB cable to PC





LED is on/off ???

GND

VDDIO

Unconnected!!!

Board Configurator v0.4.0

nRF54LM20 DK
001051844153

CONFIGURATION ABOUT

Write config

Load default config

BOARD CONTROLLER INFO

DK Hardware Revision: 0.2.1
Board Controller FW version: 1.2.1

PIN CONFIGURATION

6	swd-control	OFF	ON
20	vcom0-hwfc	OFF	ON
22	vcom1	OFF	ON
23	vcom1-hwfc	OFF	ON
42	vcom0	OFF	ON
45	ledcontrol	OFF	ON
47	ldo1-control	OFF	ON

/ = active low

PMIC CONFIGURATION

1	vddio	1800 mV
2	vdd-dbgr	1800 mV

Show Config JSON

Connect port VCOM0

VCOM0 HWFC autodetect lines

Connect port VCOM1

VCOM1 HWFC autodetect lines

LED Power

Disable LEDs to use the GPIOs for other purposes.

GPIO LED

Software Debugger (SWD)

Disable to use an external debugger on the target chip.

Power to P15

Enable 5.0-V power supply to P15.

VDDIO (nPM VOUT1) and VDD_DBGR (nPM VOUT2)

Voltage 1800 mV

1.8V 2.5V 3.3V

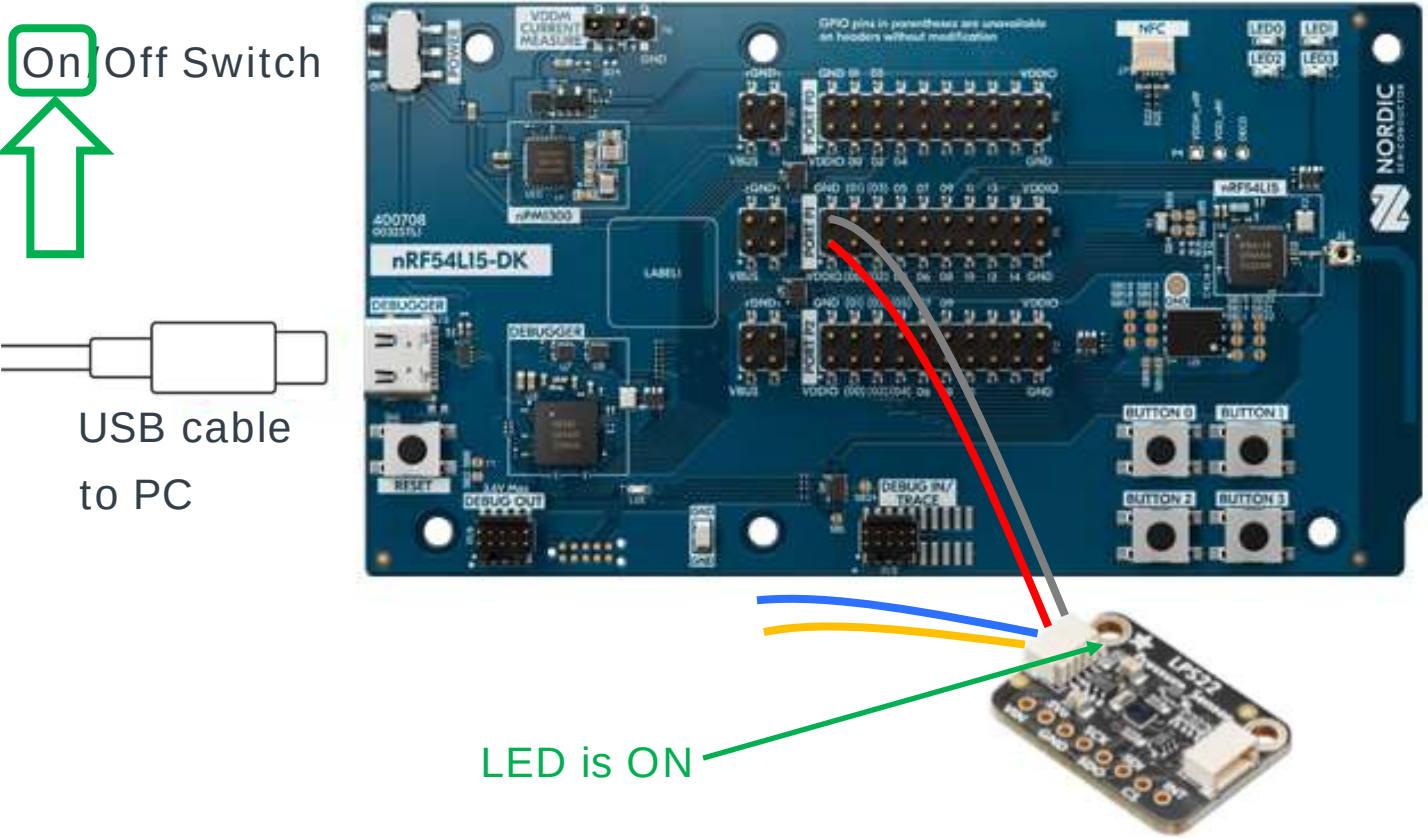
❷ Click *Write config* button

❶ Select 3.3V

```
08:08:43.050  Selecting device with the serial number 001051844153
08:08:43.054  Rendering for nRF54L20 PDK (rev. 0.3.0)
08:08:43.055  Rendering for nRF54L20 PDK (rev. 0.3.0)
08:08:44.058  Selected device with the serial number 001051844153
08:08:44.628  Rendering for nRF54L20 PDK (rev. 0.2.0)
08:08:45.175  Rendering for nRF54L20 PDK (rev. 0.2.0)
08:10:16.123  Reset to default
08:10:21.365  Reset to default
```

SHOW SIDE PANEL CLEAR LOG OPEN LOG FILE AUTOSCROLL LOG SHOW LOG

Hands-on: Connect Sensor Board



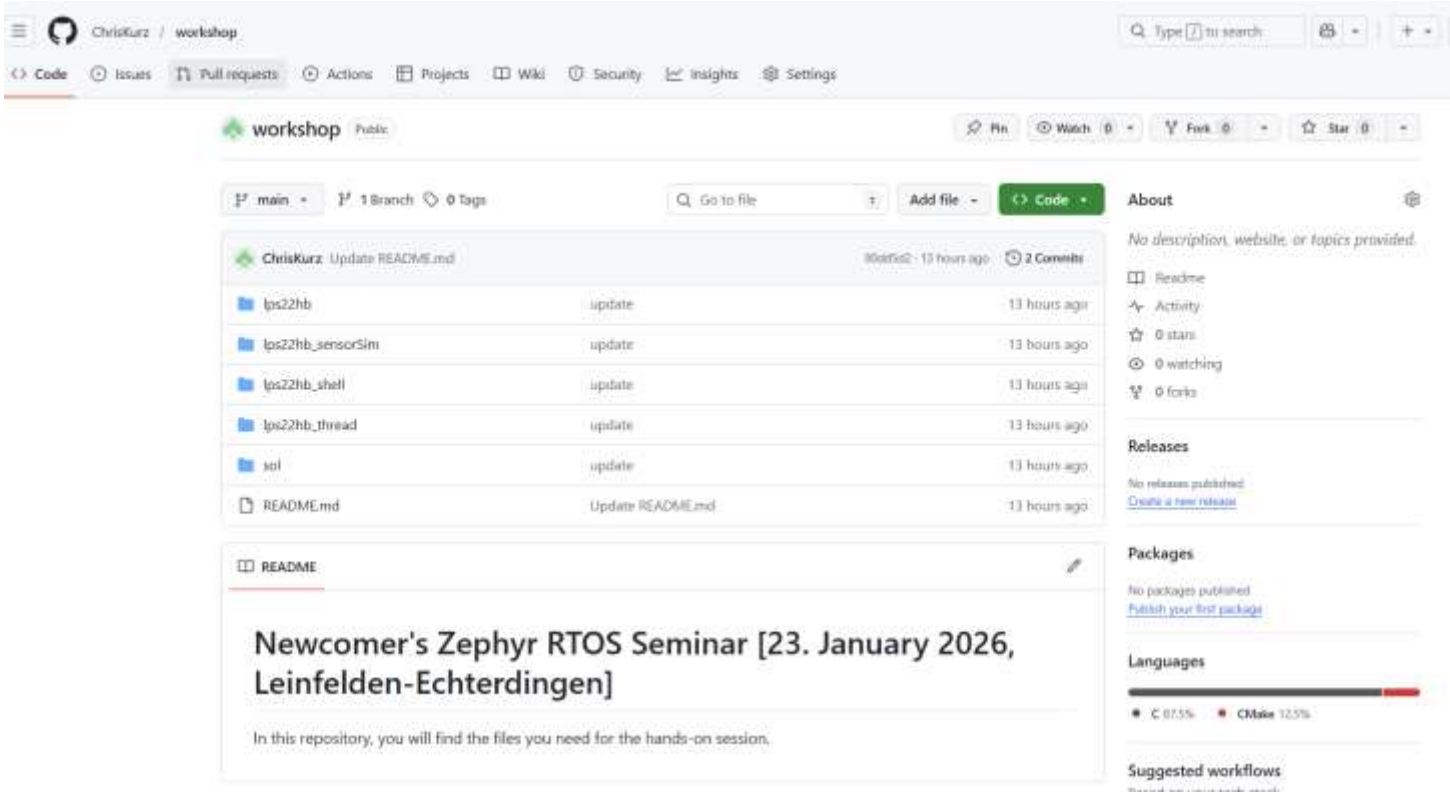
GND

VDDIO

Unconnected!!!

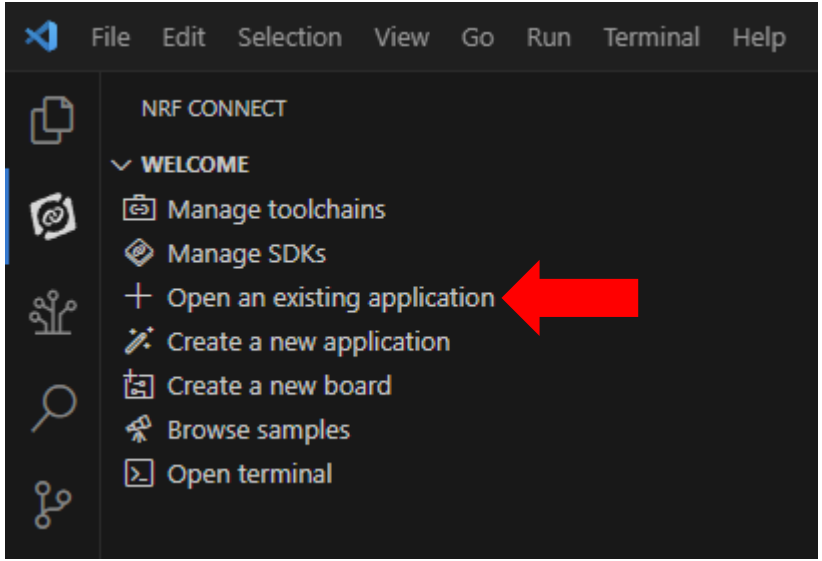
Download lps22hb projects

github.com/ChrisKurz/workshop

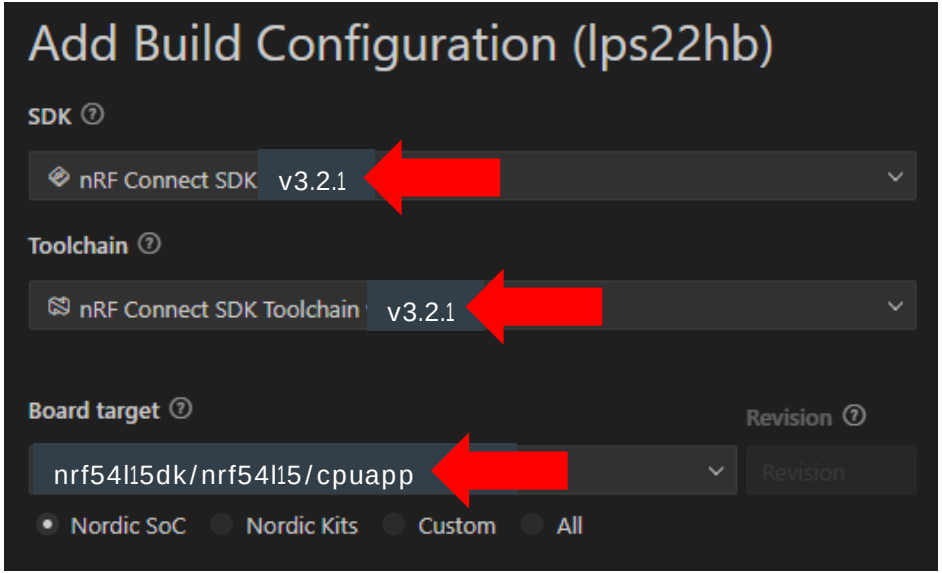


Open Existing Application

- VS Code:
  Click on „Open an existing Application“ in nRF Connect extention

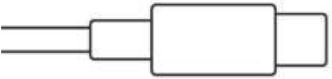


- „Add Build Configuration“
  SDK and Toolchain version: v3.2.1
  Board: nrf54l15dk/nrf54l15/cpuapp

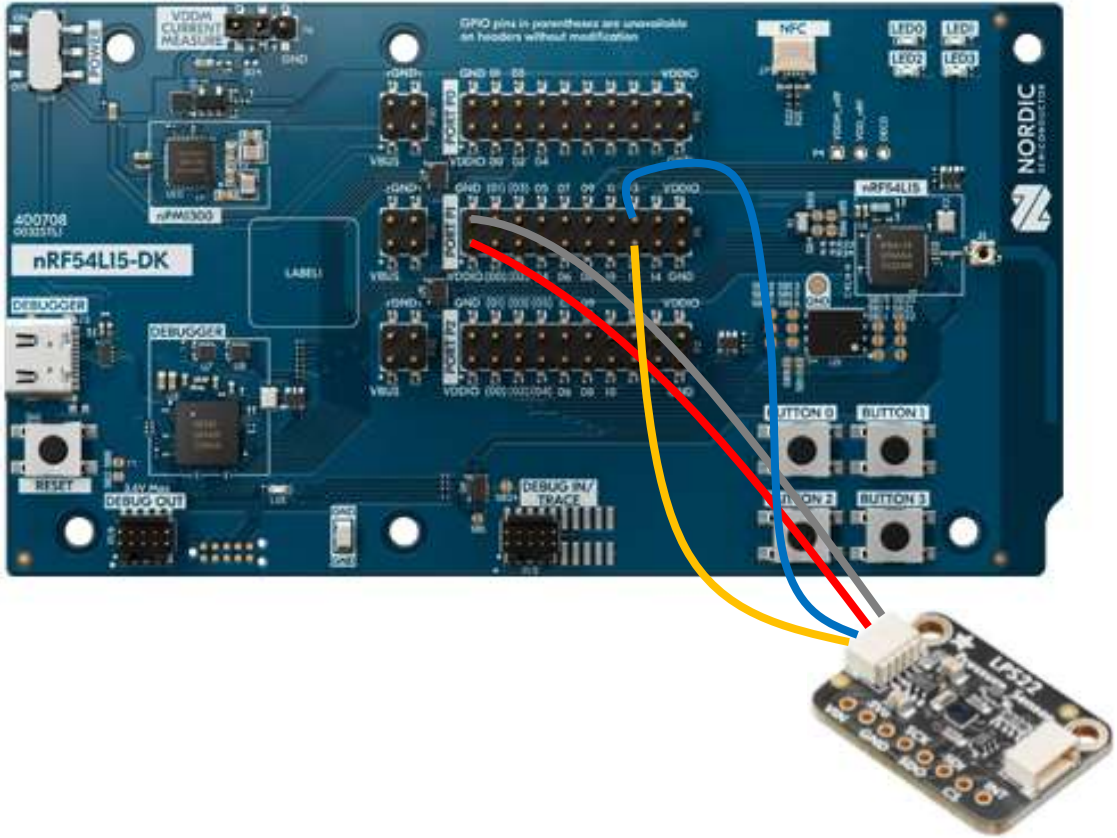


Hands-on: Connect Sensor Board

On/Off Switch



USB cable to PC



Review: main.c

Get Hardware Configuration from DeviceTree

Check if DeviceTree data is valid

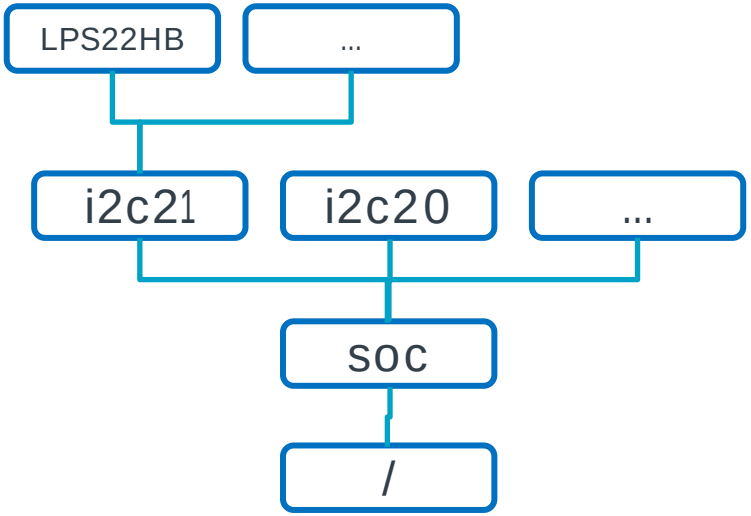
Function is handling measurement

Wait for 2 seconds

```c

int main(void)
{
    printk("LPS22HB Demo\n");

    #define SENSOR_NODE DT_NODELABEL(lps22hb)
    const struct device *const dev = DEVICE_DT_GET(SENSOR_NODE);
    if (!device_is_ready(dev)) {
        printk("Sensor is not ready %s\n", dev->name);
        return 0;
    }
    printk("Sensor is ready!\n");

    while (true) {
        process_sample(dev);
        k_sleep(K_MSEC(2000));
    }
    return 0;
}

```

Review: DeviceTree

DeviceTree Overlay file




```
&i2c21 {
        status = "okay";
        pinctrl-0 = <&i2c21_default>;
        pinctrl-1 = <&i2c21_sleep>;
        pinctrl-names = "default", "sleep";
        clock-frequency = <I2C_BITRATE_STANDARD>;

        lps22hb: lps22hb@5d{
                compatible = "st,lps22hb-press";
                reg = <0x5d>;
        };
};

&pinctrl {
        i2c21_default: i2c21_default {
                group1  {
                        psels = <NRF_PSEL(TWIM_SCL, 1, 12)>,
                                <NRF_PSEL(TWIM_SDA, 1, 13)>;
                };
        };

          i2c21_sleep: i2c21_sleep {
                group1  {
                        psels = <NRF_PSEL(TWIM_SCL, 1, 12)>,
                                <NRF_PSEL(TWIM_SDA, 1, 13)>;
                        low-power-enable;
                };
        };
};
```

Review: main.c

Get Hardware Configuration from DeviceTree

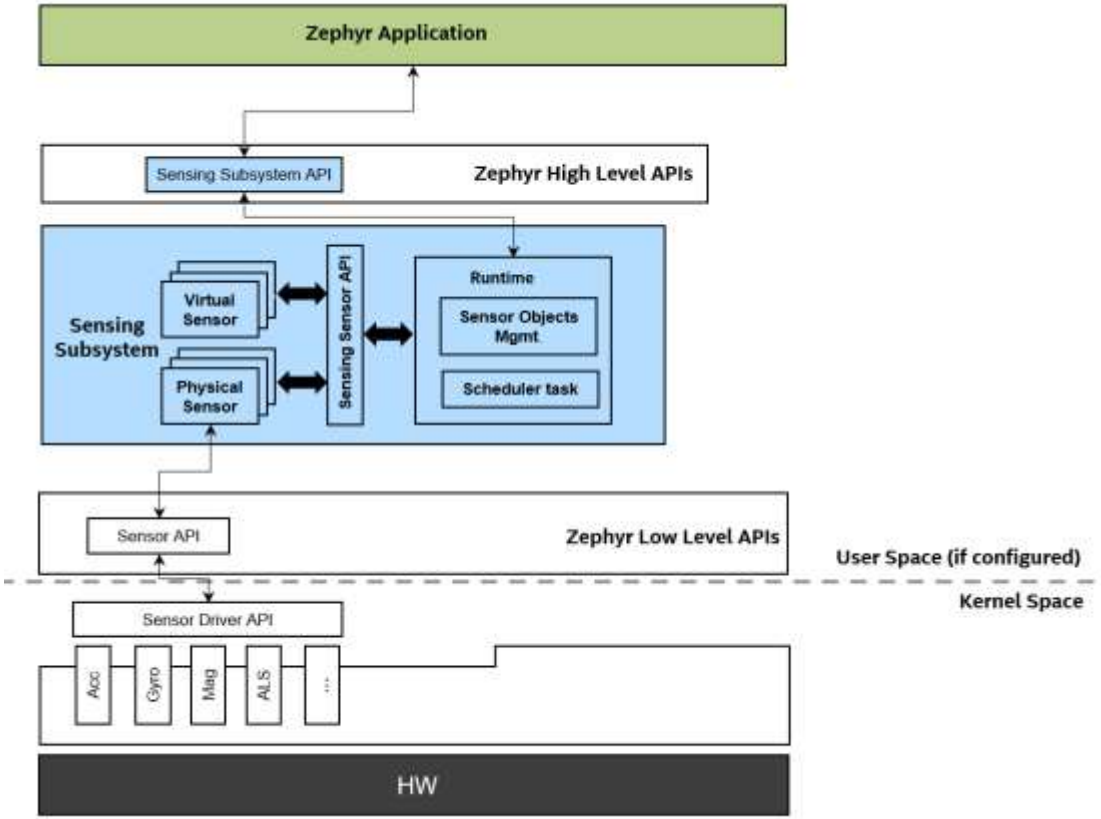
Check if DeviceTree data is valid

Function is handling measurement

Wait for 2 seconds

```c

int main(void)
{
    printk("LPS22HB Demo\n");

    #define SENSOR_NODE DT_NODELABEL(lps22hb)
    const struct device *const dev = DEVICE_DT_GET(SENSOR_NODE);
    if (!device_is_ready(dev)) {
        printk("Sensor is not ready %s\n", dev->name);
        return 0;
    }
    printk("Sensor is ready!\n");

    while (true) {
        process_sample(dev);
        k_sleep(K_MSEC(2000));
    }
    return 0;
}

```

Zephyr's Sensing Subsystem



Review: main.c

Trigger to start a measurement

Read result for pressure measurement

Read result for temperature measurement

Output Results in Serial Terminal

```c
/*
 * Copyright (c) 2025 Nordic Semiconductor ASA
 *
 * SPDX-License-Identifier: LicenseRef-Nordic-5-Clause
 */

#include <zephyr/kernel.h>
#include <zephyr/drivers/sensor.h>


static void process_sample(const struct device *dev)
{
    struct sensor_value pressure, temp;

    // Fetch a sample from the sensor and store it in an internal driver buffer
    if (sensor_sample_fetch(dev) < 0) {
        printk("Error: Sensor sample update error\n");
        return;
    }

    // Get a reading from a sensor device (read pressure)
    if (sensor_channel_get(dev, SENSOR_CHAN_PRESS, &pressure) < 0) {
        printk("Error: Cannot read LPS22HB pressure channel\n");
        return;
    }

    // Get a reading from a sensor device (read temperature)
    if (sensor_channel_get(dev, SENSOR_CHAN_AMBIENT_TEMP, &temp) < 0) {
        printk("Error: Cannot read LPS22HB temperature channel\n");
        return;
    }

    /* display pressure */
    printk("pressure: %d.%d kPa, ", pressure.val1, pressure.val2);
    //printk("Pressure: %.1f kPa, ", sensor_value_to_double(&pressure));

    /* display temperature */
    printk("Temperature: %.1f C\n", sensor_value_to_double(&temp));
}


```

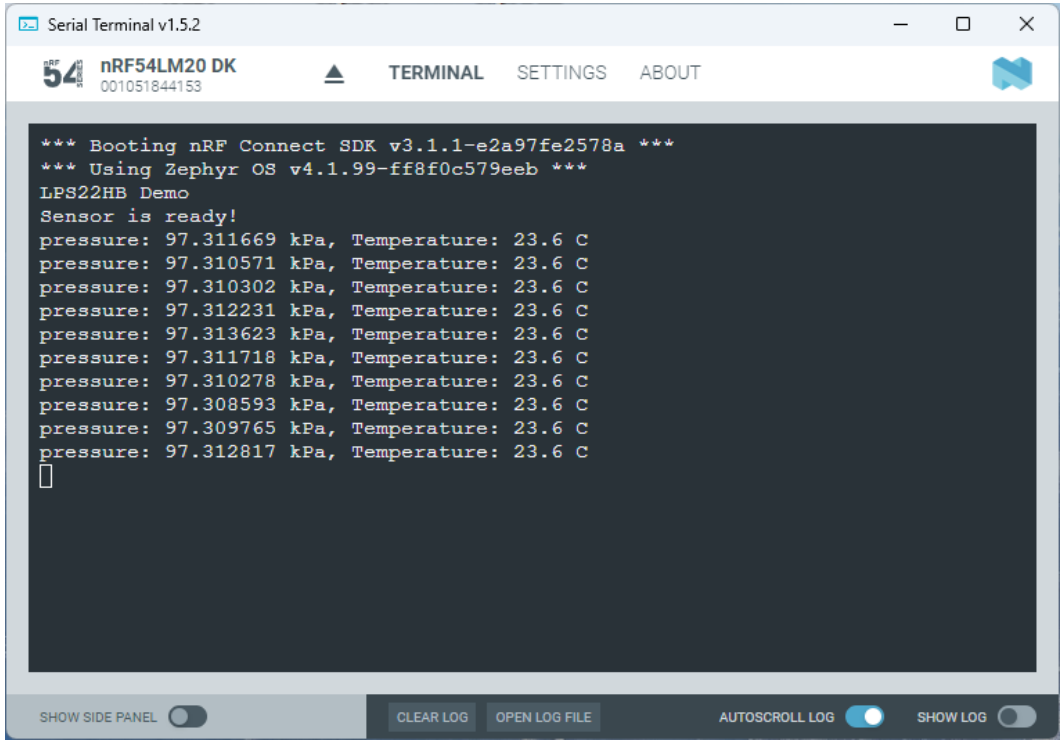
Review: prj.conf

- Enable I2C Liberary
- Enable SENSOR Library

lps22hb > prj.conf

```
CONFIG_STDOUT_CONSOLE=y
CONFIG_I2C=y
CONFIG_SENSOR=y
CONFIG_CBPRINTF_FP_SUPPORT=y
```

Serial Terminal Output



Add I2C Shell to the Project

Add these lines in prj.conf file:

```
# Enable Zephyr Shell and I2C Shell
CONFIG_SHELL=y
CONFIG_I2C_SHELL=y
```

You may comment out the line that sends conversion results to Serial Terminal

Shell Commands:

```
uart:~$ dev> device list
```

```
- i2c@c7000 (READY)
  DT node labels: i2c21
```

```
uart:~$ i2c s> i2c scan i2c@c7000
```

From LPS22HB datasheet:

Table 16. Registers address map

Name	Type	Register Address	Default	Function and comment
		Hex	Binary	
WHO_AM_I	R	0F	10110001	Who am I

I2C Slave Address

```
uart:~$ i2c read i2c@c700> i2c read i2c@c7000 5d 0f
```

Copyright © 2025 Nordic Semiconductor. All rights reserved

Using Nordic's *simulated sensor* Driver

Add following lines in the file nrf54l15dk/nrf54l15/cpuapp.overlay

```
/ {
    sensor_sim: sensor_sim{
       compatible = "nordic,sensor-sim";
       acc-signal = "wave";
       base-temperature = < 15 >;
       base-pressure = < 90 >;
    };
};
```

Change SENSOR_NODE definition to **sensor_sim**

```
#define SENSOR_NODE DT_NODELABEL(sensor_sim)
```